

Relazione - Versione 1: Baseline Agent

Introduzione

La Versione 1 è la prima versione funzionante del progetto Building Agentic AutoML.

L'obiettivo è costruire un agente AutoML minimale che riceva dataset e target, riconosca il task, addestrì un baseline e restituisca uno stato strutturato.

Questa versione si concentra sul flusso fondamentale:

```
Dataset + target
↓
Rilevare task e feature
↓
Preparare dati e baseline
↓
Valutare
↓
Restituire lo stato
```

L'architettura è volutamente semplice: un solo modello, un train-validation split e nessuna ottimizzazione. Lo scopo è comprendere il comportamento di base prima di introdurre componenti più avanzati.

Obiettivo della Versione 1

L'obiettivo della Versione 1 è creare un agente minimale ma funzionante.

L'agente deve essere in grado di:

- caricare dataset e target;
- distinguere classificazione e regressione;
- identificare feature numeriche e categoriche;
- selezionare modello e metrica;
- addestrare, valutare e restituire lo stato.

Questa versione non è ancora un sistema AutoML completo: è la base del progetto.

Architettura

Il flusso della Versione 1 è:

```
Dataset + Target → Task/Features → Preparation → Model/Metric → Evaluation → State
```

In questa versione, la funzione agent coordina direttamente tutte le operazioni.

Rilevamento, preparazione, scelta del modello, valutazione e stato sono quindi ancora concentrati nello stesso flusso.

Rilevamento del task

Il primo passo decisionale consiste nell'analizzare la colonna target.

La regola iniziale è intenzionalmente semplice: se il target contiene al massimo 20 valori unici, il problema viene considerato di classificazione; altrimenti viene considerato di regressione.

```
def detect_task(df, target):  
    n_unique = df[target].nunique()  
    if n_unique <= 20:  
        return "classification"  
    return "regression"
```

Sul dataset binario del notebook il risultato è:

```
"classification"
```

Rilevamento delle feature

L'agente separa le feature numeriche da quelle categoriche utilizzando direttamente i dtype del dataframe.

Nel dataset di classificazione vengono rilevate:

```
numerical = ['num_1', 'num_2', 'num_3', 'num_4']  
categorical = ['category_1', 'category_2']
```

Questa scelta evita configurazioni manuali nella prima versione e permette al flusso di adattarsi automaticamente alla struttura tabellare.

Preparazione dei dati

LightGBM può gestire direttamente le variabili categoriche. Per questo l'agente converte le colonne categoriche nel dtype pandas category e separa le feature X dal target y.

```
for column in categorical_features:  
    data[column] = data[column].astype("category")  
X = data.drop(columns=target)  
y = data[target]
```

Modello baseline

Il modello viene scelto in funzione del task rilevato.

```
classification → LGBMClassifier  
regression     → LGBMRegressor
```

Entrambi utilizzano random_state=42 e la configurazione di default di LightGBM. Non viene ancora effettuata alcuna ricerca di iperparametri.

Selezione della metrica

Anche la metrica viene scelta automaticamente in base al task:

```
classification → ROC AUC  
regression     → RMSE
```

In questo modo lo stesso agente può usare una misura coerente senza richiedere una configurazione manuale aggiuntiva.

Addestramento e valutazione

L'agente divide il dataset in training e validation con `test_size=0.2` e `random_state=42`.

Per la classificazione utilizza anche `stratify=y`, così da mantenere la distribuzione delle classi nei due sottoinsiemi.

Dopo il fit, la valutazione dipende dal task:

```
classification: predict_proba → roc_auc_score
regression:      predict      → RMSE
```

Sul dataset binario incluso nel progetto, il notebook ottiene:

```
model = LGBMClassifier
metric = roc_auc
score = 0.9068167604752971
```

Il risultato conferma che il baseline è già operativo sul dataset di esempio.

Agent

Nella Versione 1, l'Agent esegue le seguenti operazioni:

1. carica dataset e target;
2. rileva classificazione o regressione;
3. identifica e prepara le feature;
4. seleziona modello e metrica;
5. addestra e valuta il baseline;
6. restituisce lo stato finale.

Uno stato tipico appare così:

```
{
  "task": "classification",
  "model": "LGBMClassifier",
  "metric": "roc_auc",
  "score": 0.9068167604752971
}
```

Lo stato rende esplicite le principali decisioni e il risultato della valutazione.

Test di regressione

Lo stesso agente viene verificato anche su un secondo dataset, modificando soltanto il file di input.

L'agente rileva autonomamente il nuovo task e produce:

```
task = regression
model = LGBMRegressor
metric = rmse
score = 1.8760451113860066
```

Il test mostra il comportamento AutoML di base: modello e metrica cambiano senza modificare la logica dell'agente.

Cosa insegna la Versione 1

La Versione 1 insegna la logica fondamentale di un agente AutoML.

L'idea principale è:

osservare → decidere → preparare → addestrare → valutare → restituire

Il notebook mostra come un processo di machine learning possa essere trasformato in una sequenza di decisioni automatiche coordinate da un unico agente.

Introduce inoltre il concetto di stato come rappresentazione compatta delle decisioni e dei risultati prodotti dal sistema.

Limitazioni

La Versione 1 presenta alcune limitazioni naturali:

- il rilevamento del task usa soltanto il numero di valori unici del target;
- viene utilizzata una sola famiglia di modelli, LightGBM;
- non esiste confronto tra più algoritmi;
- non esiste ricerca degli iperparametri;
- non esistono strategie di preprocessing oltre alla conversione delle categorie;
- la valutazione usa un solo train-validation split;
- non esiste cross-validation;
- non esiste gestione avanzata di missing value, outlier o leakage;
- non esiste ancora un planner o un sistema multi-agent.

Queste limitazioni sono intenzionali.

Definiscono il percorso di evoluzione delle versioni successive.

Conclusione

La Versione 1 è la base del progetto Building Agentic AutoML.

Crea il primo agente funzionante capace di passare da un dataset tabellare a un modello baseline valutato, adattandosi automaticamente tra classificazione e regressione.

La lezione più importante è che l'AutoML agentico non consiste soltanto nell'addestrare un modello.

È un sistema che osserva i dati, prende decisioni sul task e sulle feature, sceglie gli strumenti di modellazione appropriati, valuta il risultato e restituisce uno stato interpretabile.

Il passo successivo naturale è ampliare le capacità decisionali dell'agente introducendo componenti e strategie aggiuntive per rendere il processo AutoML progressivamente più autonomo e robusto.